Imitation Learning for Autonomous Driving in CARLA

Jordy Kieto

Code, data samples and policy: https://github.com/JordyKieto/carla-imitation-learning
Training curves: results/training_curves.ipynb · Video: https://youtu.be/rr_uS4bf0B4

Abstract

Behavioral cloning trains a policy offline, under the expert's state distribution, but deploys it autoregressively: every action the policy takes shapes the observations it receives next. We study how much closed-loop driving competence a compact multimodal policy can acquire from offline expert demonstrations in the CARLA simulator, and how the design of the demonstration distribution affects what emerges under deployment. The policy fuses 5-frame histories of RGB images, a LiDAR bird's-eye-view histogram, vehicle telemetry and lane waypoints through per-modality encoders and a two-layer temporal Transformer, and regresses continuous throttle, brake and steering at 20 Hz. Demonstrations went through three stages: a small manually collected set, broader curated recordings, and finally a *systematic route-generation* procedure. That procedure enumerates spawn points, queries the road topology for feasible maneuvers, and keeps autopilot rollouts only after verifying the maneuver actually executed. The released policy (1.36 M parameters) was trained on 236,882 windows (≈3.3 h) from 448 such captures. Despite the simplicity of offline behavioral cloning, the resulting policy drives autoregressively, alone on the road and without human intervention, for hours on both training routes and held-out routes. In the author's runs it did so without collisions, and it transfers qualitatively to an unseen CARLA town whose road geometry differs from its demonstration towns. We additionally observe recovery from large trajectory deviations, although we do not claim systematic recovery without controlled evaluation. We report measured offline metrics, reconstruct the provenance of the released checkpoint from logged artifacts, and state explicitly which closed-loop findings are observed rather than measured. All code, the checkpoint (Keras and ONNX), a data sample and a claim-by-claim evidence audit are released.

1 Introduction

End-to-end driving policies trained by imitation are attractive because they turn a hard control problem into supervised learning: record an expert, then regress its actions from its observations [1, 2]. The difficulty is that the supervised objective and the deployment setting disagree. Training minimizes error on states the *expert* visited. Deployment runs the *learner* in a loop, where its own small errors move it into states the expert never demonstrated. Those errors can compound [4]. Low validation loss is therefore necessary but not sufficient evidence of driving ability. The object of interest is the closed-loop behavior.

This report documents a completed imitation-learning experiment built around that distinction. Rather than proposing a new architecture, we use a deliberately compact multimodal temporal policy as an instrument. The question we ask is:

> *How much closed-loop driving competence can a multimodal behavioral-cloning policy acquire from offline expert demonstrations, and how does the coverage of the demonstration distribution shape the behaviors that emerge under autoregressive deployment?*

The project progressed from a tiny demonstration set to systematic, verified route generation. This amounts to an intervention on the *support* of the demonstration distribution, which is the quantity behavioral cloning is fundamentally limited by.

Contributions.

1. **A systematic demonstration-generation procedure** for CARLA. It enumerates spawn points, derives the feasible maneuvers at the next junction from the road graph, randomizes start poses, and accepts an autopilot rollout only after classifying the maneuver it actually executed. It produces a maneuver-balanced set of short, reproducible, labeled captures (§5).
2. **A reproducible behavioral-cloning pipeline and released policy.** It covers sensor rig, data contract, preprocessing, window indexing and training, plus the trained checkpoint in Keras and ONNX, with provenance tied to a logged training run (§4, §6, Appendix A).
3. **An evidence-graded account of closed-loop behavior** (hours of autonomous driving on training and held-out routes, zero-shot transfer to an unseen town, recovery from deviations), with each claim classified as measured, observed, inferred or unverified (§7 and the released claim audit).

2 Related Work

End-to-end driving from demonstrations. ALVINN [8] and NVIDIA's PilotNet [2] showed that a network can map camera images directly to steering. PilotNet already recognized the covariate-shift problem and augmented training with synthetically shifted and rotated views to teach recovery. Our policy follows this lineage but is multimodal and temporal, and it deliberately contains **no synthetic recovery augmentation** (§7.4).

CARLA and conditional imitation. CARLA [1] was introduced with an imitation-learning baseline among its reference agents, so our experiment sits inside one of the simulator's original research purposes. Conditional imitation learning [3] resolves the ambiguity of junctions by conditioning the policy on a navigational command. Our policy instead receives a local lane-waypoint path. As we show in §4.1, that path does not always disambiguate junctions, which makes this a weaker form of conditioning than [3].

Covariate shift and data aggregation. Ross and Bagnell [9] showed that behavioral cloning's cost can grow quadratically with horizon, and DAgger [4] fixes this by querying the expert on learner-visited states. Prakash et al. [6] studied which aggregated states matter for vision-based urban driving in CARLA. DART [7] injects noise into the *expert's* demonstrations so that recovery states appear in the data. Our route generator's randomized start poses also introduce limited off-nominal states into the demonstrations. This is conceptually related to expert-noise approaches such as DART, although the perturbations were designed for route diversity rather than recovery training (§5.2). We account for it when interpreting recovery.

Privileged teachers. Learning by Cheating [5] trains a privileged agent on ground-truth simulator state and distills it into a sensorimotor student. Our expert, the CARLA Traffic Manager autopilot, is also privileged. Unlike [5], our student *also* receives privileged route context at test time, so we state explicitly what information crosses that boundary (§4.1, §8).

3 Problem Formulation

An expert policy π^* produces a demonstration dataset

$$\mathcal{D} = \{(o_t, a_t)\}_{t=1}^{N}, \qquad a_t = (\tau_t,\ b_t,\ \delta_t) \in [0,1] \times [0,1] \times [-1,1], \tag{1}$$

of throttle τ, brake b and steering δ at 20 Hz. The policy observes a temporal stack of $K = 5$ frames (0.25 s) across four modalities,

$$o_t = \left(I_{t-K+1:t},\ L_{t-K+1:t},\ s_{t-K+1:t},\ W_{t-K+1:t} \right), \tag{2}$$

where I is RGB, L a LiDAR bird's-eye-view (BEV) histogram, s telemetry and W lane waypoints, all in the ego frame.

Offline objective. Behavioral cloning fits π_θ by empirical risk minimization under the *expert* state distribution d_{π^*}:

$$\theta^* = \arg\min_\theta \ \mathbb{E}_{(o,a)\sim d_{\pi^*}} \left[\ell(\pi_\theta(o), a) \right], \qquad \ell(\hat{a}, a) = H_1(\hat{\tau} - \tau) + H_1(\hat{b} - b) + (\hat{\delta} - \delta)^2, \tag{3}$$

with the Huber loss $H_1(x) = \frac{1}{2}x^2$ for $|x| \le 1$ and $|x| - \frac{1}{2}$ otherwise.

Closed-loop deployment. At test time, actions feed back into the simulator,

$$o_t \xrightarrow{\pi_\theta} \hat{a}_t \xrightarrow{\text{CARLA}} o_{t+1} \xrightarrow{\pi_\theta} \hat{a}_{t+1} \longrightarrow \cdots, \tag{4}$$

so the learner is evaluated under its *own* distribution $d_{\pi_\theta} \neq d_{\pi^*}$. If ϵ is the learner's per-step error rate under d_{π^*}, the expected cost over a horizon T is bounded only as [9]

$$J(\pi_\theta) \ \le \ J(\pi^*) + O(T^2\epsilon), \tag{5}$$

because a single mistake can place the learner in states with no demonstrations. At 20 Hz a one-minute drive is $T = 1200$ decisions. Held-out action error measures only ϵ under expert states; it does not by itself establish driving competence, which must ultimately be assessed closed-loop (e.g. by route completion and infractions, as in CARLA's own benchmarks [1]).

4 Multimodal Temporal Policy

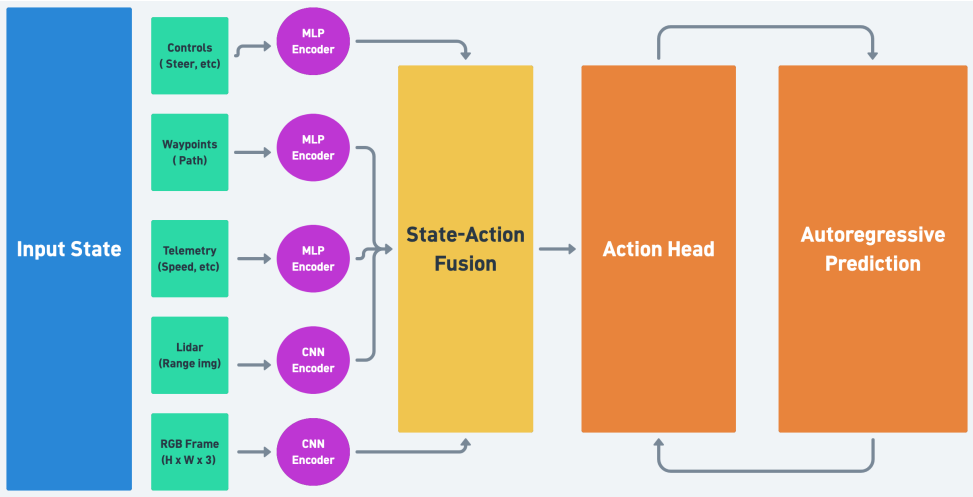


Figure 1: Conceptual overview of the policy. The released checkpoint's inputs are RGB, LiDAR BEV, telemetry and waypoints; past controls are not an input (§4.2).

Table 1: Observation modalities.

Modality	Construction	Shape
RGB I_t	Forward camera, FOV 90°, mounted at $(0.4, 0, 1.6)$ m; captured at 224^2, bilinearly resized to 96^2, scaled to $[0, 1]$	$96 \times 96 \times 3$
LiDAR L_t	32-channel ray-cast LiDAR, 50 m range, 50k pts/s; BEV histogram (Eq. 6)	$64 \times 64 \times 1$
Telemetry s_t	Soft-saturated kinematics (Eq. 7)	5
Waypoints W_t	25 lane-center waypoints spaced 2 m (Eq. 8)	25×3

4.1 Observations

LiDAR BEV. For points (x_i, y_i) in the sensor frame with $0 < x_i < 30\,\text{m}$ and $|y_i| < 10\,\text{m}$,

$$H_{jk} = \sum_i \mathbb{1}\big[(x_i, y_i) \in \text{bin}_{jk}\big], \qquad L_t = \frac{H}{\max_{jk} H_{jk} + 10^{-6}}. \tag{6}$$

Telemetry. With world-frame velocity v, acceleration α and yaw ψ, forward and lateral speeds are $v_f = v_x \cos\psi + v_y \sin\psi$ and $v_\ell = -v_x \sin\psi + v_y \cos\psi$, and

$$s_t = \Big(\tanh\tfrac{\|v\|}{20},\ \tanh\tfrac{3.6\|v\|}{72},\ \tanh\tfrac{\|\alpha\|}{5},\ \tanh\tfrac{v_f}{20},\ \tanh\tfrac{v_\ell}{20}\Big). \tag{7}$$

The first two components are identical by construction ($3.6/72 = 1/20$), so the policy effectively receives four distinct telemetry signals.

Waypoints (route context). Starting from the lane waypoint nearest the ego vehicle, successive waypoints $w_{k+1} = \text{next}_{2\,\text{m}}(w_k)$ are expressed in the ego frame, $p_k = T_{\text{ego}}^{-1} w_k = (x_k, y_k)$, with $d_k = \|p_k\|$, and encoded as

$$W_t[k] = \Big(\tfrac{x_k}{d_k},\ \tfrac{y_k}{d_k},\ \tfrac{\min(d_k, 50)}{50}\Big), \qquad k = 0, \ldots, 24. \tag{8}$$

Two properties matter for interpretation. (i) Waypoints come from the simulator's HD-map lane graph and the **ground-truth ego pose**, which makes them privileged information. (ii) At a junction $\text{next}(\cdot)$ has several successors and the generator takes the first. The waypoint path therefore follows the lane graph rather than the expert's chosen maneuver, and can disagree with it before a turn.

4.2 Architecture

Per timestep, each modality has its own encoder with weights shared across time:

$$z_t = \big[\, f_{\text{rgb}}(I_t);\ f_{\text{lidar}}(L_t);\ g_w(W_t);\ g_s(s_t) \,\big] \in \mathbb{R}^{128+128+32+32}, \tag{9}$$

where f_{rgb} and f_{lidar} are three strided 3×3 convolutions (32, 64, 128 channels, ReLU) with global average pooling, and g_w, g_s are single dense layers (32 units, ReLU). The fused features are projected and position-encoded,

$$h_t^{(0)} = \text{ReLU}(W_p z_t) + e_t \in \mathbb{R}^{256}, \tag{10}$$

then processed by two post-norm Transformer blocks [10] (4 heads, key dimension 64, feed-forward 512):

$$\tilde{h} = \text{LN}\big(h + \text{MHA}(h, h, h)\big), \qquad h' = \text{LN}\big(\tilde{h} + W_2\,\text{ReLU}(W_1 \tilde{h})\big). \tag{11}$$

The action is decoded from the most recent timestep, $\hat{a}_t = W_o\,\text{ReLU}(W_h\, h_t^{(2)})$. The model has **1,358,211 parameters**. At inference, throttle and brake are clipped to $[0, 1]$ and steering to $[-1, 1]$.

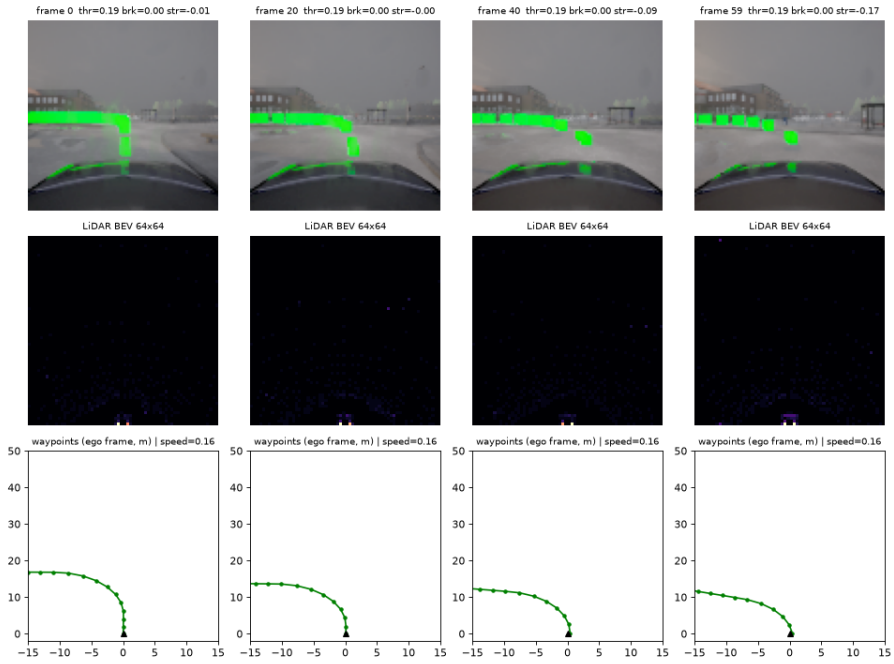


Figure 2: Four frames (3 s) of processed training data entering a left turn: RGB (top), LiDAR BEV (middle), and the waypoint path in metres in the ego frame (bottom). The green points in the RGB frames are debug waypoints drawn in the simulator during collection; they are *visible to the camera* (§8).

Positional embedding. Inspecting the serialized checkpoint shows that e_t is not a trainable parameter. The embedding layer was applied to a constant index tensor, so the framework evaluated it once at construction and stored its randomly initialized output as a constant. Temporal order is therefore still encoded, but by a fixed random code rather than a learned one. We report the model as it was actually trained.

We do not claim architectural novelty. The design is a compact, conventional late-fusion temporal policy chosen so that the scientific variables are the data and the closed-loop behavior.

5 Demonstrations

5.1 Expert and label construction

The expert is either a human driver (keyboard) or the CARLA Traffic Manager autopilot, which acts on privileged simulator state. The simulator runs in synchronous mode at $\Delta t = 0.05\,\text{s}$. Camera and LiDAR messages are paired only if their frame indices differ by at most one and their timestamps by at most $1.5\,\Delta t$. Otherwise the frame is dropped rather than recorded misaligned.

Keyboard control is binary, so recorded labels are conditioned before storage. Throttle and steering pass through an exponential low-pass filter and a rate limiter:

$$\bar{u}_t = \alpha\, u_t + (1-\alpha)\,\bar{u}_{t-1}, \qquad \alpha = \frac{\Delta t}{\tau_u + \Delta t}, \tag{12}$$

$$u_t^{\text{label}} = u_{t-1}^{\text{label}} + \text{clip}\big(\bar{u}_t - u_{t-1}^{\text{label}},\ -r_u^{\downarrow}\Delta t,\ r_u^{\uparrow}\Delta t\big), \tag{13}$$

with $\tau_{\text{throttle}} = 0.5\,\text{s}$, $r^{\uparrow} = 1.0/\text{s}$, $r^{\downarrow} = 1.5/\text{s}$ and a 0.03 deadband, and $\tau_{\text{steer}} = 0.2\,\text{s}$ with $r = 2.5/\text{s}$. Brake is recorded raw. The learning target is therefore a smooth, physically plausible control signal rather than a bang-bang keypress.

5.2 From small demonstrations to systematic route generation

The demonstration set evolved in three stages.

1. **Minimal manual demonstrations.** Early experiments used a very small set of manual demonstrations. The author reports that roughly seven minutes of driving already produced a policy that followed the road in Town01, with about one in ten rollouts ending in a collision. *These figures are author-reported; no run or dataset from this stage is part of the released artifact.* The qualitative lesson that drove the next stages was that recognizable closed-loop behavior can emerge from very little data, but the failures cluster where the data is thin.
2. **Broader curated demonstrations** extended coverage to hours of driving.
3. **Systematic route generation** (`collect.py`, first committed 2026-05-05) replaced ad hoc collection with a controlled procedure. This procedure produced the released policy's training data.

Route generation. Autopilot demonstrations have a subtle flaw: at an intersection the autopilot's choice is stochastic, so turns are under-represented and uncontrolled. The generator makes coverage a design variable:

- **Enumerate** every spawn point p on the map.
- **Query feasible maneuvers.** Walk the lane graph forward in 2 m steps (up to 40) to the first branching point. For each successor branch, compare the heading 16 m further along with the heading at the branch, $\Delta\psi = \mathrm{wrap}(\psi_{\text{branch}} - \psi_{\text{ref}})$, and label the maneuver $m =$ `left` if $\Delta\psi > 25°$, `right` if $\Delta\psi < -25°$, else `straight`.
- **Randomize the start.** Sample $\Delta\psi_0 \sim \mathcal{U}(-3°, 3°)$, lateral offset $\sim \mathcal{U}(-0.25, 0.25)$ m and longitudinal offset $\sim \mathcal{U}(-0.5, 0.5)$ m.
- **Roll out** the autopilot for 30 s (600 frames) at a fixed 18 mph target speed, with lane changes disabled.
- **Verify, then accept.** Classify the executed maneuver from the ego yaw trajectory,

$$\hat{m} = \mathrm{cls}\Big(\max_t \big|\mathrm{wrap}(\psi_t - \psi_0)\big| \Big),$$

  with the same 25° threshold applied to the signed extremum. Keep the capture only if $\hat{m}$ is feasible at this spawn point and its quota (captures per maneuver) is not yet filled. Retry up to 20 times per spawn point.

The verify-then-accept step turns a stochastic expert into a *controlled* data source. Every stored capture carries a verified maneuver label, the spawn index, the start-pose perturbation and sensor-alignment statistics. The randomized start also has a second-order effect: each capture begins slightly off the lane center and heading. This introduces limited off-nominal states, conceptually related to expert-noise approaches such as DART [7], although the perturbations were designed for route diversity rather than recovery training. We return to this in §7.4.

5.3 The training set

We reconstructed the released policy's training set from logged artifacts (Appendix A); Table 2 summarizes it.

Table 2: Training set of the released policy.

Quantity	Value
Maps	**Town01, Town02**
Recordings	**448** (394 on 2026-05-09, 54 on 2026-05-10)
Median recording length	595 windows ≈ 600 frames = 30 s (route generator capture length)
Five-frame windows	**236,882** ≈ **3.3 h** of 20 Hz driving
Direction mix (by window)	straight 89.47%, left 5.85%, right 4.67%
Windows with brake > 0	2.0%
Mean target throttle / mean $\|\delta\|$	0.235 / 0.030

Windowing. Each frame gets a direction label, `left` if $\delta \leq -0.08$, `right` if $\delta \geq 0.08$, `straight` otherwise. A window $[t-4, t]$ is kept only if its majority direction covers at least 60% of its frames. This filter removes ambiguous transition windows so that the per-direction diagnostics in §6 are well defined. It also means some turn-entry transitions are excluded from training.

Even with maneuver-balanced *captures*, the balance by *window* is dominated by straight driving (89%), because a 30 s capture spends most of its frames approaching and leaving the junction. This is a concrete example of why demonstration coverage has to be measured at the unit the loss actually sees.

6 Training and Offline Evaluation

6.1 Protocol

The model was trained with Adam (learning rate 3×10^{-3}, batch size 409) on the loss of Eq. 3. Windows were split *per recording and per direction*: the first 70% of each recording's windows of a given direction went to training and the remaining 30% to validation, giving 165,777 training and 71,108 validation windows with matching direction ratios. Every 50 steps the model was evaluated on 50 validation batches. The run lasted 53 epochs (21,750 steps, 8.88 M windows, 1.64 h), and checkpoints were written every 10 epochs. **The released policy is the epoch-10 checkpoint.**

This split is a *temporal* hold-out within recordings. It measures interpolation to unseen moments of seen routes, not generalization to unseen routes or maps. We therefore use it only to diagnose fitting, never as evidence of driving ability.

6.2 Metrics

Beyond per-control losses and MAE, we logged diagnostics aimed at driving-relevant failure modes:

- **Turn detection.** With $\mathcal{P} = \{t : |\hat{\delta}_t| > 0.1\}$ and $\mathcal{G} = \{t : |\delta_t| > 0.1\}$, we report precision $|\mathcal{P} \cap \mathcal{G}|/|\mathcal{P}|$, recall $|\mathcal{P} \cap \mathcal{G}|/|\mathcal{G}|$ and their harmonic mean F_1.
- **Turn onset delay** $\min \mathcal{P} - \min \mathcal{G}$, in frames.
- **Lagged error** $\frac{1}{n-k} \sum_t |\hat{a}_t - a_{t+k}|$ for $k \in \{1, 2, 3\}$, the error against targets k frames in the future. Error that shrinks with k would indicate anticipation of upcoming controls; error that grows with k indicates predictions tracking the current control.
- **Smoothness** $\frac{1}{n-1} \sum_t |\hat{a}_{t+1} - \hat{a}_t|$.
- **Per-direction MSE**, to expose the class imbalance of §5.3.

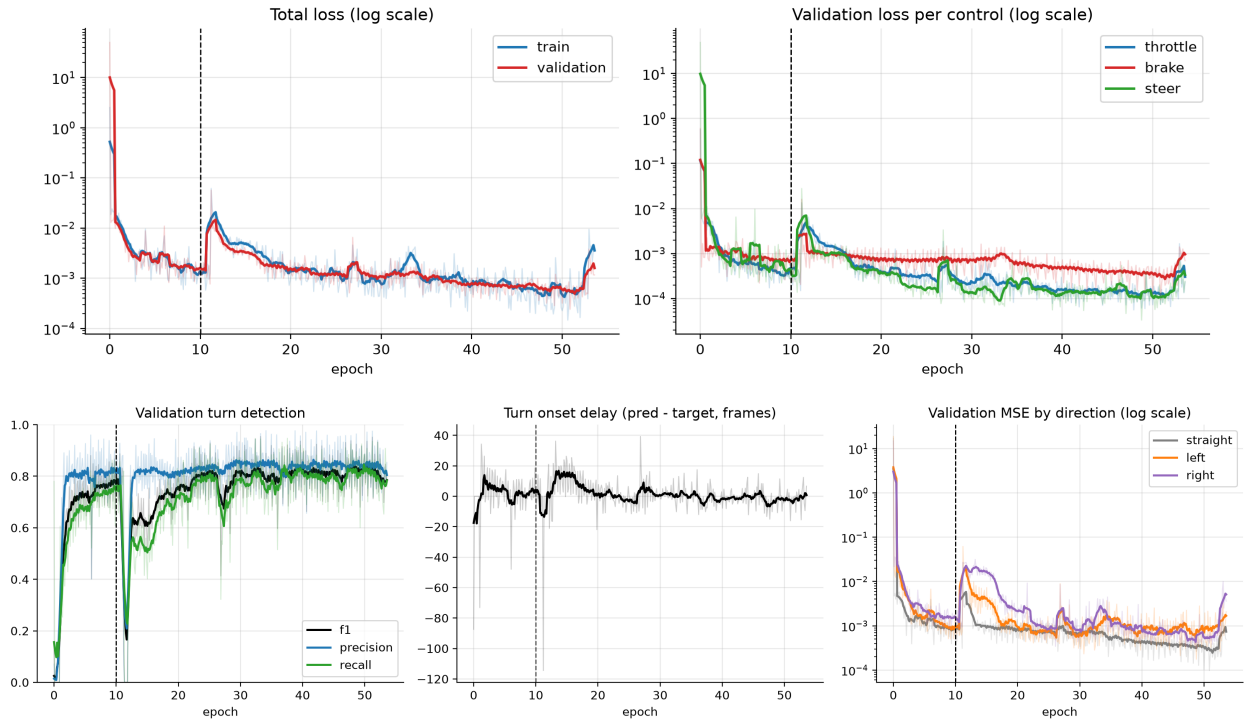


Figure 3: Training curves of the full run, regenerated from the exported metric history. Top: total train/validation loss and per-control validation loss. Bottom: validation turn detection, turn onset delay and per-direction validation MSE. Faint lines are raw evaluations, solid lines a 9-point moving average; the dashed line marks the released epoch-10 checkpoint. Source: `results/training_curves.ipynb`.

Table 3: Offline validation metrics. Epoch 1: last evaluation of the epoch. Other rows: mean over all validation evaluations within the stated epochs.

		MAE				Val MSE by direction		
	Val loss	throttle	brake	steer	Turn F_1	straight	left	right
Epoch 1 (end)	1.26e-2	0.072	0.009	0.056	0.39	5.8e-3	9.3e-3	1.3e-2
Epoch 10 (released)	**1.53e-3**	**0.017**	**0.017**	**0.013**	**0.78**	**0.84e-3**	**0.98e-3**	**1.57e-3**
Epochs 50–53	0.70e-3	0.008	0.004	0.006	0.82	0.36e-3	0.93e-3	0.83e-3

6.3 Results

Figure 3 and Table 3 support four observations.

1. **Most of the offline fit happens early.** From epoch 10 to epoch ≥ 50 the validation loss roughly halves again. Turn F_1, the metric closest to a driving decision, only moves from 0.78 to 0.82.
2. **Turns remain the hard case.** Throughout training, right-turn validation error stays about 2× straight-driving error, and by epochs 50–53 left-turn error is about 2.6× straight. This matches the 5–6% share of turn windows, and it is exactly where closed-loop failures would be expected.
3. **Per-batch R^2 is uninformative here.** Logged R^2 values are extremely negative (e.g. -3.5×10^3 for steering at epoch 10) even though MAE is around 0.01. Validation batches are contiguous slices of recordings, so the target variance *within* a batch is often near zero and $R^2 = 1 - \mathrm{SS}_{\mathrm{res}}/\mathrm{SS}_{\mathrm{tot}}$ blows up. We report this so the metric is not misread, and recommend dataset-level R^2 in future runs.
4. **Training briefly destabilized right after the released checkpoint.** Shortly after epoch 10, train and validation loss spike by more than an order of magnitude (validation loss peaks at 5.7×10^{-2}, 37× the epoch-10 mean) and turn F_1 collapses to 0, before smoothed validation loss and turn F_1 return to their epoch-10 levels around epochs 20 and 22 (Figure 3). The released policy therefore precedes this episode. Its cause was not investigated.

Offline, a steering MAE of ≈0.013 means the policy predicts the expert's steering closely *on expert states*. By §3, this does not by itself say the policy can drive.

7 Closed-Loop Behavior

7.1 Deployment protocol

The policy runs at 20 Hz in synchronous mode. At each tick the latest aligned sensor frame is encoded, pushed into a 5-frame buffer (padded by repetition after a reset), and passed to the policy. The clipped action is then applied and the simulator is advanced. No expert, action filtering or safety controller intervenes. The policy's actions alone determine its future observations. Evaluation maps, spawn points and traffic settings are selected in the driving tool (`drive.py`).

7.2 Sustained autoregressive driving

Here *autoregressive* means the policy alone controls the vehicle: it is the only agent on the road, no expert or human intervenes, and every observation it receives is a consequence of its own previous actions. Under these conditions the released policy drove for **hours**, on both **training routes and held-out routes**, following lanes and negotiating turns, with **no collisions and no human intervention** in the author's runs (video). At 20 Hz an hour is $T = 72{,}000$ consecutive self-conditioned decisions. Within this traffic-free setting, lane following and turning were effectively solved at the level of observation.

These runs were not logged as a benchmark (no fixed route set, route-completion or infraction counts), so this is a strong observation rather than a measurement. It also says nothing about traffic, pedestrians or traffic rules, which were absent.

Evidence class: qualitative observation (author-run, hours of driving); not yet quantified.

7.3 Driving on unfamiliar road geometry

All demonstrations were collected on Town01 and Town02. Both are small towns whose roads are two-lane streets with T-junctions. Figure 4 shows the policy in a town outside that set, driving a multi-lane curved

Table 4: Proposed protocol for quantitative closed-loop evaluation (not yet run). Each condition would use a fixed set of spawn points and routes, several seeds, and no human intervention.

Condition	Maps	Metrics
In-distribution	Town01, Town02	route completion, collisions per km, off-road events per km, interventions
Unseen towns	any town other than Town01/Town02	same as above
Recovery from offsets	all of the above	recovery rate and time-to-recover (§7.4)

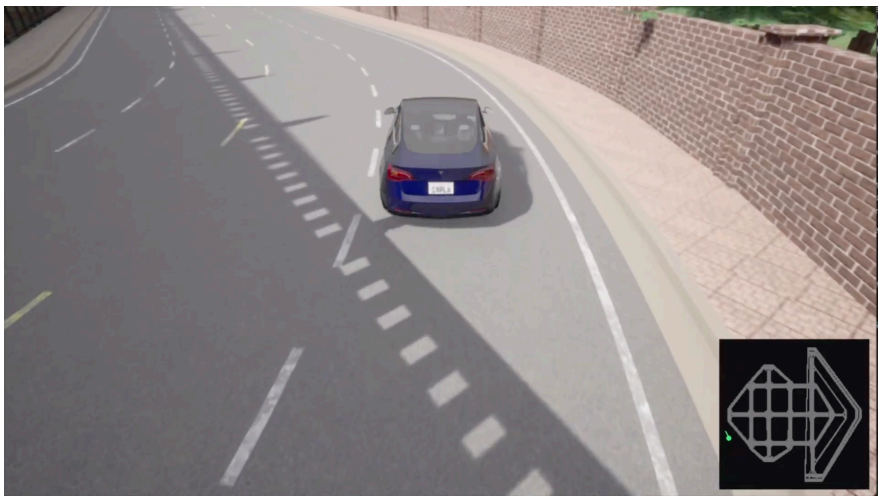


Figure 4: The policy driving a curved multi-lane road in an unseen CARLA town. All demonstrations were collected on Town01 and Town02. The minimap (bottom right) shows the map layout.

road with different markings, curvature and surroundings. No demonstrations were recorded in this town, so this is an instance of **zero-shot transfer to an unseen CARLA town**.

A plausible interpretation is that the policy learned a *local* conditional driving function,

$$\pi\big(a_t \mid \text{visual road geometry},\ \text{local occupancy},\ \text{ego kinematics},\ \text{local route}\big), \tag{14}$$

rather than memorizing map-specific appearance or positions: none of its inputs encodes global position or map identity. We state this as an interpretation, not a demonstrated mechanism. It is also consistent with the policy relying heavily on the privileged waypoint path, which is available on any map (§8). Separating these explanations requires input ablations (§9).

Evidence class: qualitative observation in an unseen town; not yet a systematic measurement (Table 4).

7.4 Recovery from large deviations

In behavioral cloning the textbook failure is a drift into a state the expert never visited, followed by further errors. During closed-loop runs the author observed the policy leaving the lane or road, steering back toward drivable surface, reacquiring the lane and resuming route following.

Recovery evaluation protocol (proposed). Recovery can be measured directly by initializing the policy at a controlled offset from the lane center. Given a lane waypoint with pose (x_0, y_0, ψ_0), spawn the vehicle at

$$(x, y, \psi) = \big(x_0 - \Delta y\ \sin\psi_0,\ \ y_0 + \Delta y\ \cos\psi_0,\ \ \psi_0 + \Delta\psi\big), \tag{15}$$

with lateral offsets $\Delta y \in \{\pm 1, \pm 2, \pm 3\}$ m and heading offsets $\Delta\psi \in \{0°, \pm 15°, \pm 30°\}$, i.e. up to 12× the lateral and 10× the heading perturbation present in the demonstrations (§5.2). A rollout counts as a **recovery** if, within $T_r = 5$ s and without collision, the vehicle reaches $|e_y| < 0.5$ m and $|e_\psi| < 5°$ relative to the lane and holds this for 1 s. Reporting the recovery rate and time-to-recover as a function of $(\Delta y, \Delta\psi)$ turns the qualitative observation into a curve showing where recovery breaks down.

What the data rules out, and what it does not. The development pipeline later gained a dedicated *recovery-route* generator. We verified that its data (4,350 windows, recorded 2026-05-17) was collected *after* the released checkpoint (2026-05-10) and is **not** in the training set. No online DAgger recordings are in the training set either. The observed recoveries are therefore not imitated from recovery or learner-state demonstrations. The one remaining source of off-nominal states is the route generator's randomized start poses (±0.25 m, ±3°), which begin every capture slightly perturbed, although far smaller than the deviations recovered from. A natural hypothesis is that the autopilot's corrections from these small perturbations, combined with the LiDAR and waypoint geometry, generalize to larger deviations. That is an inference to be tested, not a finding.

Evidence class: qualitative observation; a recovery mechanism is not claimed. The protocol above would quantify it.

8 Limitations and Threats to Validity

- **Evidence strength.** Closed-loop results (hours of intervention-free driving, unseen-town transfer, recovery) are author observations, not logged benchmark metrics. There are no multi-seed retrainings, no systematic rollout statistics (Table 4 is a protocol, not results) and no architecture or input ablations.
- **Privileged inputs at test time.** Waypoints come from the HD-map lane graph and the ground-truth ego pose. In addition, debug waypoints drawn during collection (and during autopilot/policy driving) are **rendered into the RGB images**, so route information also reaches the vision channel. We make no claim that the student is free of privileged information.
- **Junction ambiguity.** The waypoint path takes the first lane-graph successor, so before a turn it can contradict the expert's maneuver. Turn behavior is therefore not cleanly conditioned on intent.
- **Evaluation split.** Validation is a temporal hold-out within the training recordings.
- **Data composition.** 89% straight driving by window. There is no traffic, no pedestrians and no weather variation, and the autopilot ignored lights and signs during route generation. Demonstrations cover only two maps (Town01, Town02). Map tags were recorded for only ≈36% of windows in the manifest, so per-map data proportions are unknown.
- **Implementation details affecting interpretation.** The positional embedding is a fixed random constant (§4.2); two telemetry components are duplicates; LiDAR BEV max-normalization is dominated by near-ego returns.
- **Historical reconstruction.** Early-stage quantities (≈7 min, ≈1/10 collisions) are author-reported and not backed by released artifacts.
- **Simulation only.** No claims are made about real-world driving.

9 Future Work

1. **Systematic closed-loop evaluation** (Table 4): fixed spawn/route sets on Town01/Town02 and unseen towns, several seeds, route completion, collisions and off-road events per km.

2. **Input ablations** to separate representation from privileged context: drop waypoints, remove debug rendering from RGB, and swap in noisy or lagged route context.
3. **Demonstration-coverage scaling:** train on 7 min, 30 min, 1 h and 3.3 h subsets of the verified route captures and measure closed-loop metrics against data size, turning the project's historical progression into a controlled experiment.
4. **Recovery attribution:** run the offset-initialization protocol (§7.4) on policies trained with no start jitter, the current jitter and explicit recovery routes, and compare their recovery curves.
5. **Intent conditioning:** feed the planned route (or a command, as in [3]) instead of first-successor lane waypoints.

10 Conclusion

A compact multimodal policy trained purely offline on ≈3.3 hours of systematically generated, maneuver-verified autopilot demonstrations drives autoregressively in CARLA, alone on the road, for hours on training and held-out routes without collisions or human intervention. It also drives in an unseen CARLA town and sometimes returns to the road after large deviations, even though no recovery demonstrations were in its training data. Offline metrics show that the policy fits expert actions closely, but under autoregressive deployment that fit is only a precondition. The contribution of this report is less the network than the method: controlling and verifying the demonstration distribution, reconstructing the provenance of the released policy from logged artifacts, and grading every claim by its evidence. These observations motivate a controlled study of how demonstration coverage and privileged route context shape closed-loop competence in behavioral cloning.

The systematic demonstration pipeline emerged after earlier attempts with reinforcement learning and less structured imitation pipelines failed to produce reliable closed-loop driving. Those experiments are outside the evidentiary scope of this report.

References

[1] A. Dosovitskiy, G. Ros, F. Codevilla, A. López, and V. Koltun. CARLA: An open urban driving simulator. In *Conference on Robot Learning (CoRL)*, 2017.

[2] M. Bojarski et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

[3] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy. End-to-end driving via conditional imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[4] S. Ross, G. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[5] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl. Learning by cheating. In *Conference on Robot Learning (CoRL)*, 2019.

[6] A. Prakash, A. Behl, E. Ohn-Bar, K. Chitta, and A. Geiger. Exploring data aggregation in policy learning for vision-based urban autonomous driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[7] M. Laskey, J. Lee, R. Fox, A. Dubey, and K. Goldberg. DART: Noise injection for robust imitation learning. In *Conference on Robot Learning (CoRL)*, 2017.

[8] D. A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1989.

[9] S. Ross and J. A. Bagnell. Efficient reductions for imitation learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

A Artifact Provenance

Evidence	Value
Training run	`barzksni` ("MLP – Generalized Cross Map"), created 2026-05-10 22:00:32 UTC; full metric history in `results/barzksni/`, plotted in `results/training_curves.ipynb`
Checkpoint	`artifacts/policy.keras`, Keras 3.13.2, `date_saved` 2026-05-10 22:19:15 UTC
Link	Epoch 10 completed at 1,115 s = 22:19:07 UTC; checkpoint saved 8 s later; checkpoints every 10 epochs
Training maps	Town01, Town02
Training data	448 recordings dated 2026-05-09/10; 236,882 windows (training-log sampling summary = manifest subset, identical direction ratios)
Excluded data	`recovery`-tagged recordings (2026-05-17) postdate the checkpoint; no DAgger-mode recordings
Code timeline	MLP policy (2026-01-27) → sensor-sync and data-contract fixes (2026-03-25) → first end-to-end driving (2026-04-07) → route generator (2026-05-05) → training (2026-05-10) → "all maps" model milestone (2026-05-12)
ONNX	`artifacts/policy.onnx`, max absolute deviation from Keras $< 4 \times 10^{-7}$